BACSE202 — Database Management Systems

Fall Semester 2026–2027

DBMS Stock Market Analytics

& Portfolio Management System

Team Members

K.M. MOHAMED IRFAN 25BCE1462

EASWAR AP 25BCE1512

MOHAMMED NADEEM 25BCE1771

Faculty: Dr. S. Geetha

VIT Chennai

Problem Statement & Objectives

The Problem

Individual investors lack integrated tools combining market analytics, portfolio management, and ML-based predictions — features locked behind expensive institutional platforms.

Raw NSE stock market data is heavily denormalized — 87.7 MB CSV with company info repeated across 287,310 rows, making it a classic DBMS normalization challenge.

287,310	49	13	1,653
Price Records	Companies	Sectors	Corp. Actions

Objectives

- Design a fully normalized relational schema (1NF to BCNF) from raw CSV market data
- Build dual-database architecture — PostgreSQL for relational data + TimescaleDB for time-series
- Implement secure user authentication with bcrypt hashing and JWT tokens
- Develop paper trading with virtual cash, row-level locking, and trigger-based audit trail
- Train ML pipeline (HistGradientBoosting) for 5-day stock trend prediction
- Build a financial UI with React and interactive charts
- Achieve 72 automated tests covering all modules

System Architecture

Tech Stack

Frontend:
React 18, Vite, Recharts

Backend:
FastAPI, Python 3.12

Auth:
bcrypt 12-round, JWT

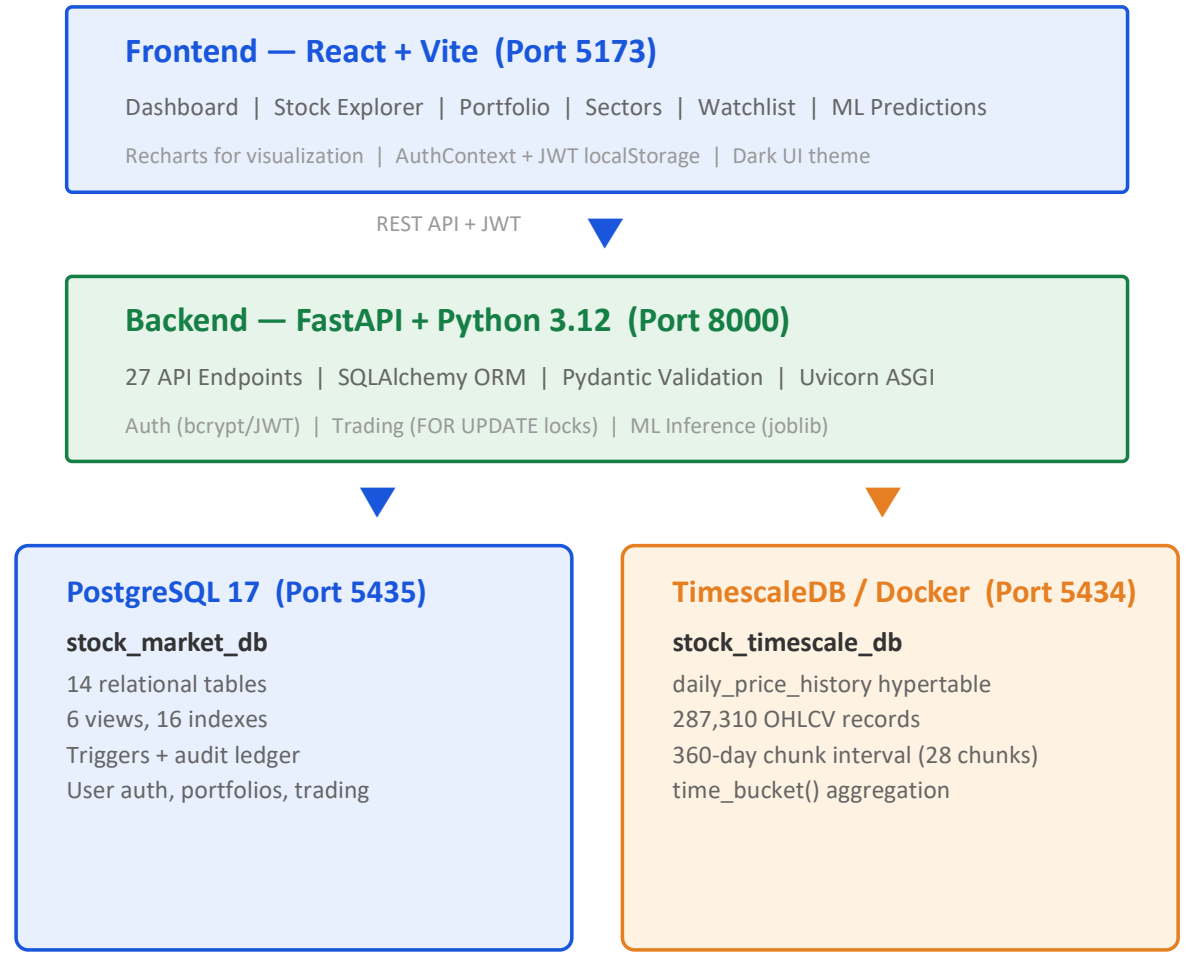
Relational DB:
PostgreSQL 17

Time-Series:
TimescaleDB (Docker)

ML:
scikit-learn, joblib

Testing:
pytest — 72 tests

DevOps:
Docker Compose

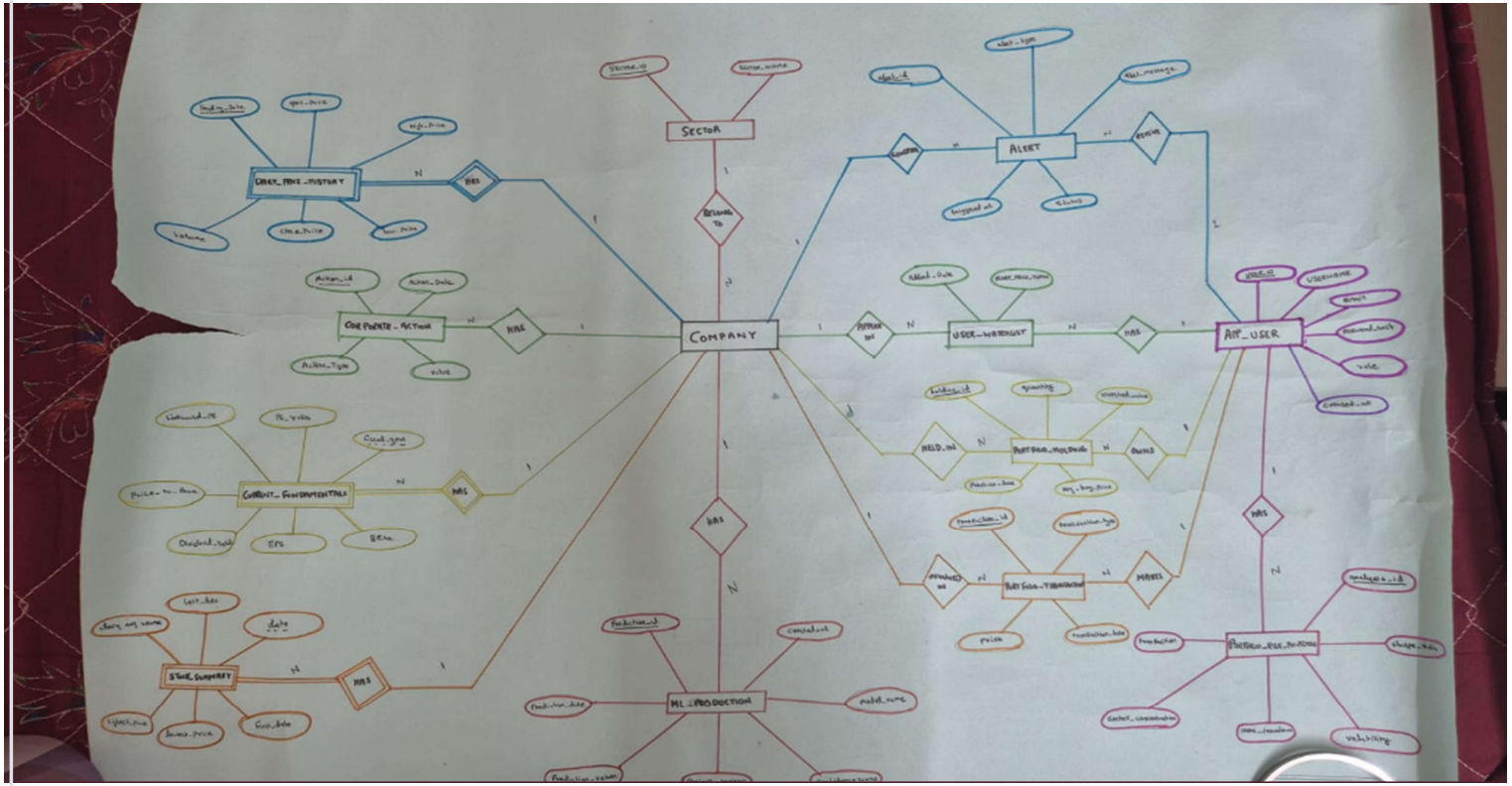


ER Diagram

16 entities | Dual-database boundary | Chen's notation with cardinality ratios

Entities (16)

sector (13 rows)
company (49)
daily_price_history (287K)
corporate_action (1,653)
current_fundamentals (49)
stock_summary (49)
app_user (137)
user_portfolio (149)
portfolio_holding (36)
portfolio_transaction (66)
portfolio_txn_audit (66)
portfolio_risk_analysis (2)
risk_sector_allocation (4)
user_watchlist (2)
alert (0)
ml_prediction (0)



Relational Schema — Keys & Constraints

sector
PK: sector_id | UK: sector_name

company
PK: ticker | FK: sector_id → sector
ON DELETE RESTRICT

daily_price_history
PK: (ticker, trading_date) | FK: ticker → company
CHECK volume ≥ 0 | TimescaleDB hypertable

corporate_action
PK: action_id | FK: ticker → company
UNIQUE(ticker, action_date, type)
CHECK type IN ('Dividend','Stock_Split')

current_fundamentals
PK/FK: ticker → company ON DELETE CASCADE

stock_summary
PK/FK: ticker | CHECK total_trading_days > 0

app_user
PK: user_id | UK: username, email
CHECK role IN ('admin','user') | bcrypt hash

user_portfolio
PK: portfolio_id | FK: user_id → app_user
UNIQUE(user_id, portfolio_name)
CHECK cash_balance ≥ 0 | DEFAULT 100000

portfolio_holding
PK: holding_id | FK: portfolio_id, ticker
UNIQUE(portfolio_id, ticker)
CHECK qty ≥ 0, invested_value ≥ 0

portfolio_transaction
PK: transaction_id
FK: holding ON DELETE SET NULL
CHECK type IN ('BUY','SELL','DIVIDEND','SPLIT')

portfolio_txn_audit
PK: audit_id | FK: transaction_id
Trigger-populated, immutable ledger

portfolio_risk_analysis
PK: analysis_id | FK: user_id → app_user

risk_sector_allocation
PK: (analysis_id, sector_id) | Both FK
CHECK pct BETWEEN 0 AND 100
Replaces JSONB → 3NF normalized

Normalization — From Raw CSV to BCNF

1NF **Atomic Values**

Raw CSV already satisfies 1NF.

Later found: JSONB sector_concentration column violated 1NF.

Fix: Decomposed into portfolio_risk_sector_allocation table.

2NF **No Partial Dependencies**

Composite key {Ticker, Date} had partial dependencies:

- Ticker → Company_Name (partial)
- Ticker → Sector (partial)

Split into: company, fundamentals, stock_summary tables.

3NF **No Transitive Dependencies**

Transitive chain found:
Ticker → Sector_ID → Sector_Name

Fix: Created independent sector table with surrogate key sector_id as FK in company table.

BCNF **Superkey on LHS**

All 8 decomposed relations verified. Every non-trivial FD X → Y has X as a superkey.

Lossless joins proven via Heath's Theorem for every decomposition.

Anomalies Eliminated

- Update: Change sector name in 1 row instead of 5,861 rows
- Insertion: Add new sector without needing any stock record
- Deletion: ON DELETE RESTRICT prevents losing sector data
- Redundancy: 7.18 MB repeated data removed (12.44% saving)

Storage Comparison

Raw CSV (denormalized):	87.7 MB
PostgreSQL heap:	30 MB
Indexes:	19 MB
Total Normalized DB:	**58 MB**

Saving: 7.18 MB redundant data eliminated (12.44%)

Important SQL Operations

TimescaleDB time_bucket() Aggregation

```
SELECT time_bucket('7 days', trading_date) AS bucket,
       AVG(close_price), SUM(volume)
FROM daily_price_history
WHERE ticker = :ticker
GROUP BY bucket ORDER BY bucket;
```

Buy Stock — Row-Level Locking (FOR UPDATE)

```
SELECT cash_balance FROM user_portfolio
  WHERE portfolio_id = :pid FOR UPDATE;

UPDATE user_portfolio
  SET cash_balance = cash_balance - :cost;

INSERT INTO portfolio_holding (...)
  ON CONFLICT (portfolio_id, ticker)
  DO UPDATE SET qty = qty + EXCLUDED.qty;
```

Trigger-Based Audit Trail

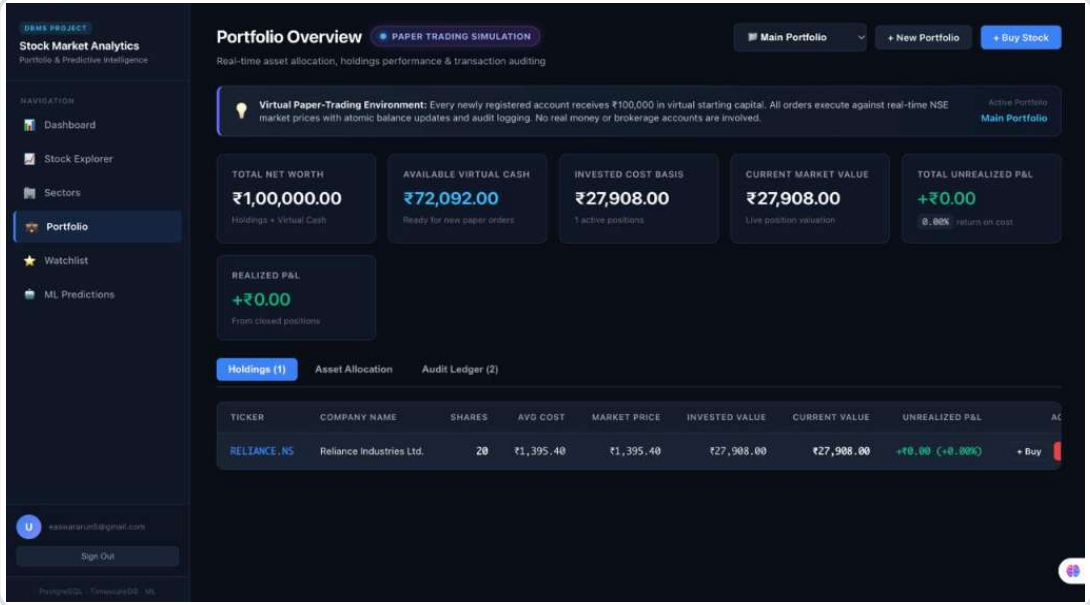
```
CREATE FUNCTION fn_audit_trigger()
RETURNS TRIGGER AS $$
BEGIN
  INSERT INTO portfolio_transaction_audit
    (transaction_id, operation,
     old_data, new_data)
  VALUES (NEW.id, TG_OP,
    row_to_json(OLD), row_to_json(NEW));
  RETURN NEW;
END; $$ LANGUAGE plpgsql;
```

Portfolio Holdings with Live P&L

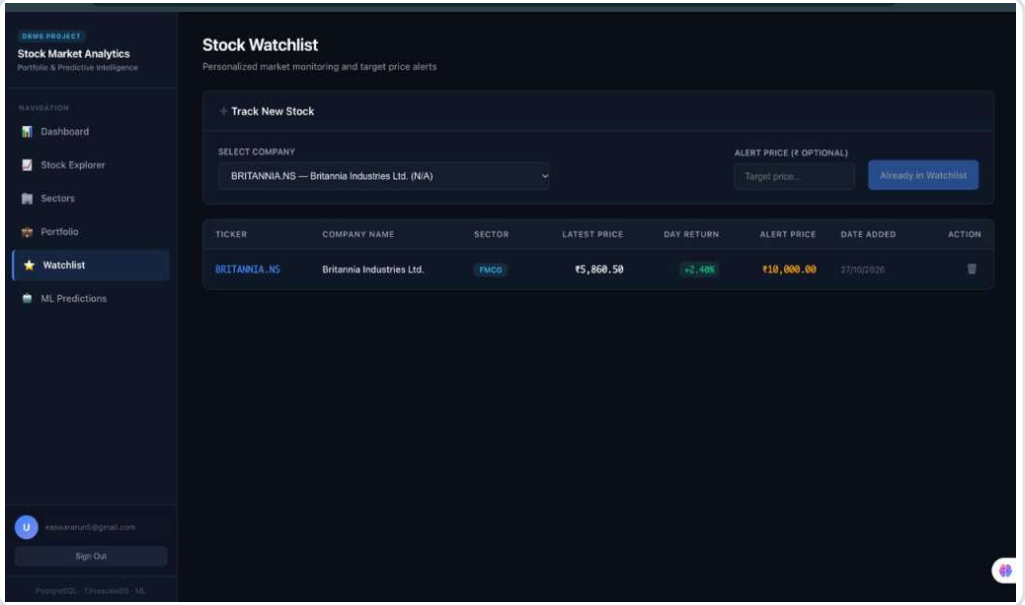
```
SELECT h.ticker, h.quantity,
  h.average_buy_price,
  lp.close_price AS current_price,
  (lp.close_price * h.quantity)
    AS current_value,
  (lp.close_price * h.quantity
    - h.invested_value) AS profit_loss
FROM portfolio_holding h
JOIN v_latest_stock_prices lp
  ON h.ticker = lp.ticker;
```

Working Application — Portfolio & Watchlist

Paper Trading Portfolio — Virtual cash, weighted avg cost, real-time P&L



Watchlist — Personalized stock monitoring with target price alerts



Portfolio Key Features

- FOR UPDATE row locks prevent double-spending
- Weighted-average cost auto-recalculated on each buy
- ON CONFLICT upsert — one row per stock per portfolio
- Every trade logged to immutable audit trail via trigger

Watchlist Key Features

- Track any Nifty 50 stock with optional alert price
- Live price & daily return from TimescaleDB
- UNIQUE(user_id, ticker) — no duplicates allowed
- User-scoped via JWT — full isolation between users

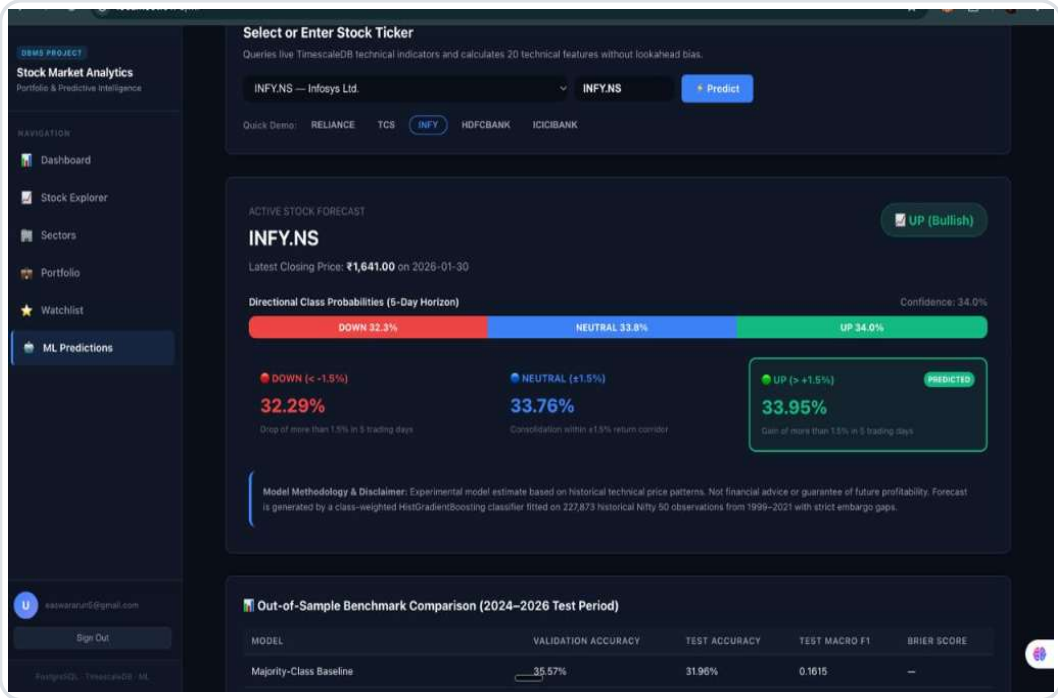
Working Application — ML Stock Trend Predictions

ML Pipeline Summary

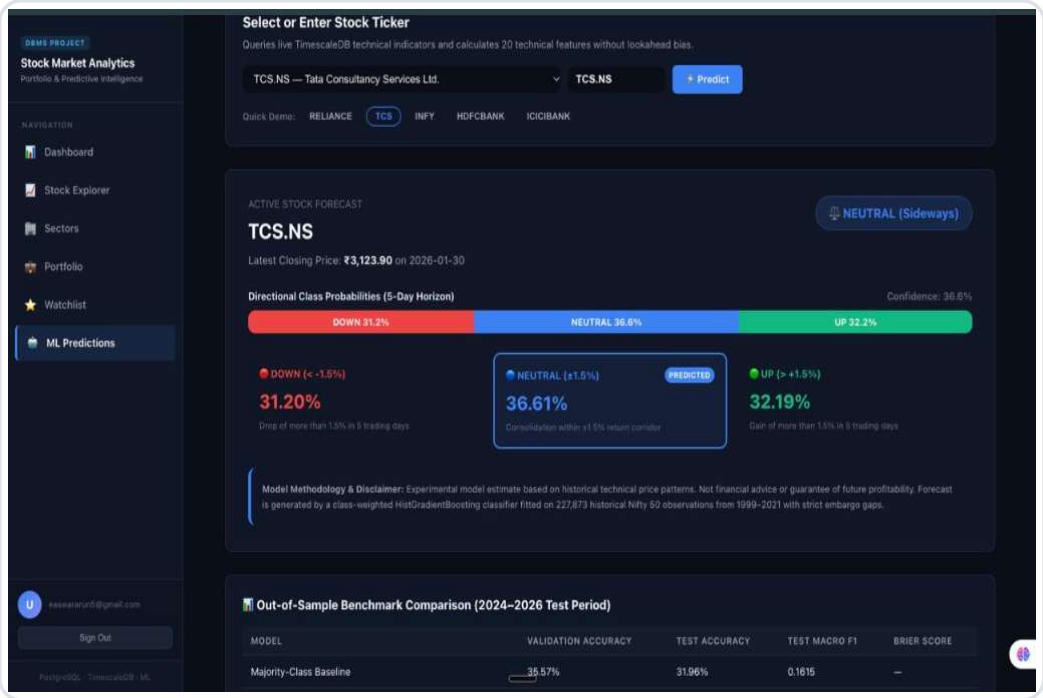
Model: HistGradientBoosting Classifier | 20 technical features (returns, MAs, volatility, RSI) | Target: 5-day trend (UP/DOWN/NEUTRAL, ±1.5% threshold)

Chronological split: Train ≤ 2021 | Validation 2022–2023 | Test ≥ 2024 | 5-session embargo gaps | 7 models compared | No data leakage

INFY.NS — Predicted: UP (Bullish), Confidence: 34.0%



TCS.NS — Predicted: NEUTRAL (Sideways), Confidence: 36.6%



Testing & Key Findings

72
Tests Passed

test_api.py	35	Health checks, dual DB, time-bucket, corporate actions, pagination
test_auth_and_portfolio.py	16	Registration, bcrypt, JWT, portfolio isolation, overselling rejection
test_ml_pipeline.py	12	No data leakage, probability normalization, chronological embargo
test_normalized_portfolio_and_risk.py	8	3NF validation, weighted-avg cost, FK integrity, audit triggers
test_paper_trading.py	1	Complete 12-step lifecycle: register → buy → sell → verify → isolation

Key Findings

- Normalization reduced storage by 12.44% — eliminated 7.18 MB of redundant data across 287,310 records
- Dual-database architecture cleanly separates OLTP (PostgreSQL) and analytical (TimescaleDB) workloads
- TimescaleDB time_bucket() provides 3–5x faster aggregation vs DATE_TRUNC on vanilla PostgreSQL
- FOR UPDATE row-level locks prevent double-spending without table-level locking
- JSONB-to-relational migration achieved 100% data integrity, verified via SHA-256 checksums
- Trigger-based audit ledger provides immutable transaction history for compliance auditing
- HistGradientBoosting with chronological embargo outperforms all baselines with zero data leakage

Limitations & Future Enhancements

Current Limitations

- Static data — prices pre-loaded up to Jan 2026, no live market feed
- Paper trading only — virtual cash, no real brokerage integration
- ML predicts 5-day trends only — no exact price targets or long-term forecasts
- Limited to 49 Nifty 50 companies — no mid-cap or international stocks
- REST polling — no WebSocket real-time streaming
- Single-server deployment — not horizontally scalable
- Alert engine schema exists but notification logic not yet built

Future Enhancements

- Live market data via NSE/Yahoo Finance APIs with scheduled ingestion
- WebSocket streaming for real-time price tickers
- LSTM/Transformer models for longer-horizon predictions
- Alert engine — email + push notifications on price triggers
- Multi-currency and international exchange support
- Horizontal scaling — read replicas, Redis cache, Kubernetes
- Mobile app (React Native) using the same backend
- Automated backtesting engine for trading strategies

Thank You!

DBMS Stock Market Analytics & Portfolio Management System